

Data Visualization

Make Data Speak With Charts That Tell Stories

■ VISUALIZATION MASTERY

■ LEARNING OUTCOME

By the end of this PDF you will create publication-quality static and interactive visualizations using Matplotlib, Seaborn, and Plotly to communicate every type of data insight.

■ Skills You Will Gain

- Create statistical charts with Seaborn
- Build custom plots with Matplotlib
- Make interactive charts with Plotly
- Choose the right chart for every insight
- Style and annotate professional visualizations
- Build multi-panel dashboards in Python

■ A data scientist who can't visualize is like a chef who can't plate food. The insight is only valuable if others can see and understand it.

SECTION 1

Visualization Library Overview

	Best For	When to Use
	Full control, custom layouts	Publication figures, custom annotation
	Statistical visualizations	EDA, distribution, regression, heatmaps
	Interactive web charts	Dashboards, hover tooltips, zoom/filter
	Declarative grammar of graphics	Concise, readable chart code
	Browser-based interactive plots	Real-time data, web embedding

SECTION 2

Matplotlib — Full Control

```
import matplotlib.pyplot as plt import numpy as np # Professional multi-panel figure fig, axes =
plt.subplots(2, 2, figsize=(14, 10)) fig.suptitle('Sales Dashboard', fontsize=16,
fontweight='bold') # Bar chart axes[0,0].bar(months, revenue, color='#7C3AED', alpha=0.8)
axes[0,0].set_title('Monthly Revenue'); axes[0,0].set_ylabel('■ Lakhs') # Line chart with
annotation axes[0,1].plot(dates, sales, color='#06B6D4', linewidth=2)
axes[0,1].axhline(sales.mean(), color='red', linestyle='--', label='Mean')
axes[0,1].annotate('Peak', xy=(peak_date,peak_val), xytext=(peak_date,peak_val+5),
arrowprops=dict(arrowstyle='->')) # Scatter with color mapping sc = axes[1,0].scatter(x,y, c=size,
cmap='viridis', alpha=0.6) fig.colorbar(sc, ax=axes[1,0]) plt.tight_layout()
plt.savefig('dashboard.png', dpi=150, bbox_inches='tight')
```

SECTION 3

Seaborn — Statistical Charts

```
import seaborn as sns sns.set_theme(style='whitegrid', palette='viridis') # Distribution:
histogram + KDE sns.histplot(df['Income'], kde=True, bins=40, color='#7C3AED') # Relationship:
scatter with regression sns.regplot(x='StudyHours', y='Score', data=df, scatter_kws={'alpha':0.4})
# Comparison: box plot sns.boxplot(x='Department', y='Salary', data=df, palette='Set2') # Heatmap:
correlation matrix sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='RdBu_r', center=0) #
Category proportions: count plot sns.countplot(x='Grade', hue='Gender', data=df, palette='husl') #
FacetGrid: same chart per group g = sns.FacetGrid(df, col='Region', row='Year')
g.map(sns.scatterplot, 'Income', 'Spend')
```

SECTION 4

Plotly — Interactive Charts

```
import plotly.express as px import plotly.graph_objects as go # Interactive scatter map fig =
px.scatter_geo(df, lat='lat', lon='lon', color='Sales', size='Revenue', hover_name='City',
title='Sales by City') fig.show() # Animated bubble chart over time fig = px.scatter(df, x='GDP',
```

```
y='LifeExp', size='Population', color='Continent', animation_frame='Year', hover_name='Country')
fig.show() # Interactive funnel chart fig = px.funnel(df, x='Count', y='Stage', title='Sales
Funnel') # Candlestick for stock data fig = go.Figure(go.Candlestick(x=df['Date'],
open=df['Open'], high=df['High'], low=df['Low'], close=df['Close']))
fig.update_layout(title='Stock Price OHLC') fig.show()
```

SECTION 5

Chart Selection Guide

Goal	Chart	Library & Code
Distribution shape	Histogram + KDE	sns.histplot(col, kde=True)
Spread & outliers	Box / Violin plot	sns.boxplot(x=cat, y=num)
Correlation	Scatter plot	sns.scatterplot(x, y, hue)
All correlations	Heatmap	sns.heatmap(df.corr(), annot=True)
Trend over time	Line chart	plt.plot(dates, values)
Category compare	Bar chart	plt.bar(cats, values)
Proportion	Pie / Donut	plt.pie(sizes, labels=labels)
Geographic	Choropleth map	px.choropleth(df, locations, color)
Feature importance	Horizontal bar	pd.Series.plot(kind='barh')
Model performance	ROC curve	sklearn roc_curve + plt.plot

SECTION 6

Professional Styling Tips

```
# Set global style plt.rcParams.update({ 'font.family': 'DejaVu Sans', 'axes.spines.top': False,
'axes.spines.right': False, 'axes.grid': True, 'grid.alpha': 0.3, 'figure.dpi': 120 }) # Color
palettes for DS SNS_PALETTES = ['viridis','plasma','Set2','husl','coolwarm'] # Always label axes
and add title ax.set_xlabel('Feature Name', fontsize=12) ax.set_ylabel('Value', fontsize=12)
ax.set_title('Descriptive Insight Title', fontsize=14, fontweight='bold') # Add value labels on
bars for bar in bars: ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.1,
f'{bar.get_height():.1f}', ha='center', va='bottom', fontsize=9)
```

■ PRACTICE QUESTIONS

Q1. When would you use Plotly instead of Seaborn?

Answer: When you need interactive charts — hover tooltips, zoom, filter, animations, or embedding in web apps. Seaborn is better for static EDA charts.

Q2. What type of chart shows the distribution of a continuous variable AND reveals the number of observations?

Answer: Histogram — bars show frequency of values in bins. Add `kde=True` to overlay the probability density curve.

Q3. You want to show how 5 features relate to each other simultaneously. What chart do you use?

Answer: Pair plot (`sns.pairplot`) — shows scatter plot for every pair of features with distribution on the diagonal.

Q4. Why should a bar chart Y-axis always start at 0?

Answer: Truncated axes make small differences look proportionally huge — misleading viewers about the actual magnitude of difference.

Q5. What does a violin plot show that a box plot doesn't?

Answer: Violin plots show the full distribution shape (KDE) on both sides, revealing multimodality and subtle distribution differences that box plots hide.

■ Visualization Cheat Sheet	
<code>sns.histplot(x, kde=True)</code>	Distribution histogram
<code>sns.boxplot(x=cat, y=num)</code>	Box plot comparison
<code>sns.violinplot(x, y, data)</code>	Violin distribution
<code>sns.scatterplot(x, y, hue)</code>	Colored scatter
<code>sns.regplot(x, y, data)</code>	Scatter + regression line
<code>sns.heatmap(corr, annot=True)</code>	Correlation heatmap
<code>sns.pairplot(df, hue)</code>	All feature pairs
<code>sns.countplot(x, hue, data)</code>	Category bar chart
<code>px.scatter(df, x, y, color)</code>	Plotly interactive
<code>px.choropleth(df, locations)</code>	Geographic map
<code>plt.tight_layout()</code>	Fix overlapping subplots
<code>plt.savefig('f.png', dpi=150)</code>	Save high-res figure